Universidade Federal de Campina Grande
Centro de Engenharia Elétrica e Informática
Departamento de Sistemas e Computação
Graduação em Ciência da Computação

Estruturas de dados lineares

1. Das estruturas de dados vistas: Pilha e Fila, escreva apenas a assinatura dos métodos de cada estrutura.
2. Um estudante de computação precisa resolver um problema bastante importante, que é executar as operações que estão descritas abaixo, cuja estrutura é uma pilha inicialmente vazia. Tão logo ele retire algum elemento desta pilha, estes deverão ser inseridos em uma fila também inicialmente vazia. A sequência de operações invocadas é: push P, push E, push R, push T, push O, pop, pop, push S, push S, push L, pop, pop, pop. Qual o estado final da fila após essa execução?
3. Em relação a estrutura de dados Fila, julgue como verdadeiras (V) ou falsas (F) as assertivas abaixo e assinale a alternativa correta:
   a. Elementos entram por uma extremidade e saem por outra
   b. Respeita a ordem temporal inversa dos elementos à medida que eles são inseridos
   c. Pode ser implementada usando-se duas pilhas
   d. Usando-se apenas os métodos da interface Fila é possível implementar uma busca na Fila em tempo constante.
4. Em relação às estruturas Pilha e Fila, assinale as assertivas abaixo e escolha a alternativa correta. Assuma que a pilha e a fila tem tamanho infinito.

   I - Filas são utilizadas quando se deseja processar itens de acordo com a ordem “primeiro-que-chega, primeiro-atendido”

   II - Uma pilha é uma estrutura linear nas quais inserções, exclusões e acessos a itens ocorrem sempre em um dos extremos da estrutura

   III - Objetos podem ser inseridos em uma pilha a qualquer momento, mas apenas o que foi inserido mais recentemente (isto é, o último) pode ser removido a qualquer momento

   IV - Em uma fila, os elementos podem ser inseridos a qualquer momento, mas apenas o elemento que está a mais tempo na fila pode ser removido

   V - Em uma fila, os elementos são inseridos e removidos de acordo com o princípio “o último que entra é o primeiro que sai"

5. Suponha que uma pilha S inicialmente vazia realizou um total 25 operações push, 12 operações top e 10 operações pop em uma determinada ordem. 3 operações pop retornaram StackUnderflowException (exceção de pilha vazia). Qual é o tamanho atual da pilha S?
6. Descreva o estado da fila (inicialmente vazia) após cada uma das seguintes operações: enqueue(5), enqueue(3), dequeue(), enqueue(2), enqueue(8), dequeue(), dequeue(), enqueue(9), enqueue(1), dequeue(), enqueue(7), enqueue(6), dequeue(), dequeue(), enqueue(4), dequeue(), dequeue().
7. Implemente duas pilhas usando um mesmo array (estrutura compartilhada) de tamanho n de forma que o overflow não acontece enquanto o número de elementos das duas pilhas juntas não ultrapassar n. As operações de push e pop devem ter complexidade O(1) .

Listas Encadeadas

1. O código abaixo representa um método de inserir no início de uma lista encadeada simples (abordagem iterativa). Observe o código e comente se ele faz realmente a inserção correta sem deixar efeitos colaterais na lista.

```java
public void insertFirst(T elem){
    SingleLinkedListNode<T> newNode = new SingleLinkedListNode<T>();
    newNode.setData(elem);
    SingleLinkedListNode<T> temp = new SingleLinkedListNode<T>();
    temp.setData(head.getData());
    newNode.setNext(temp);
    temp.setNext(head);
    head = newNode;
}
```

2. Observe os comentarios e o código abaixo e assinale a alternativa que preencheria corretamente os trechos //MISSING CODE 1 e //MISSING CODE 2, respectivamente, para o método funcionar corretamente.

```java
// A classe AscendingOrderedRecursiveLinkedListFullImpl estende
// RecursiveLinkedListImpl e representa uma lista ordenada que usa
// um comparator interno (atributo).
@Override
public void insert(T element) {
    if(isEmpty()){
        this.data = element;
        this.next = new AscendingOrderedRecursiveLinkedListFullImpl<T>(this.comparator);
    }else{
        if(comparator.compare(this.data, element) > 0){ //if data is greater
            AscendingOrderedRecursiveLinkedListFullImpl<T> newNode =
                    new AscendingOrderedRecursiveLinkedListFullImpl<T>(this.comparator);
            newNode.data = this.data;
            newNode.setNext(this.getNext());
            //MISSING CODE 1
            //MISSING CODE 2
        }else { //if data is less or equal than the element
            this.next.insert(element);
        }
    }
}
```

3. Na lista simplesmente encadeada recursiva, implemente o método de remover o último elemento da lista.

```java
public class RecursiveSingleLinkedListImpl<T> implements LinkedList<T> {

      protected T data;
      protected RecursiveSingleLinkedListImpl<T> next;
      public RecursiveSingleLinkedListImpl() { }
      public void removeLast(){
        ...
      }
}
```

4. Considere o seguinte código de lista, onde RecursiveSingleLinkedListFullImpl estende a classe RecursiveSingleLinkedListImpl. Analise o código e comente se ele funciona corretamente ou tem algum efeito colateral.

```java
public class AscendingOrderedRecursiveLinkedListFullImpl<T> extends
        RecursiveSingleLinkedListFullImpl<T> {

    /**
     * Insercao de um elemento em determinada posicao em uma lista encadeada
     * (abordagem recursiva).
     * @param elem - o elemento a ser inserido
     * @param pos - a posicao do elemento na lista. Valor que varia
     *            de 0 (inicio) até tamanho-1 (final).
     */
    public void insertAt(T elem,int pos){
        if(pos == 0){
            RecursiveSingleLinkedListFullImpl newNode =
                    new RecursiveSingleLinkedListFullImpl<T>();
            newNode.setData(this.getData());
            this.setData(elem);
            this.setNext(newNode);
            newNode.setNext(this.getNext());
        }else{
            ((AscendingOrderedRecursiveLinkedListFullImpl)this.next).insertAt(elem, pos: pos-1);
        }
    }
```

Tabelas Hash

1. Desenhe uma tabela hash com endereçamento aberto e tamanho 9 (com probing linear). Use a função de hash "f(k)=k%9". Insira as chaves: 5, 29, 20, 0, 27 e 18 na tabela nessa ordem e mostre o estado final da tabela após executar todas as operações.
2. Você planeja inserir 1000 itens em uma tabela hash e quer um número médio de 2 acessos em uma busca com sucesso. Qual o tamanho do array interno que você pode utilizar considerando que a tabela tem endereçamento fechado e usa chaining para resolver os conflitos?
3. Uma tabela hash com chaining tem tamanho 512. Qual o número máximo de elementos que podem ser colocados na tabela?
4. Suponha que você deseja fazer um estudo comparativo entre duas tabelas hash de tamanho 11. Uma delas usa endereçamento aberto com probing linear e função primária de hash h'(k)=k % m. A outra usa a mesma função primária e probing quadrático com c1=1 e c2=3. Qual a melhor tabela considerando que as operações realizadas foram apenas inserções dos valores: 10,22,31,4, 15, 28, 17, 88, 59?
5. Considere uma tabela de espalhamento (tabela hash) de comprimento m = 11, que usa endereçamento aberto (open addressing), a técnica de tentativa linear (linear probing) para resolver colisões e com a função de dispersão (função hash) h(k) = k % m, onde k é a chave a ser inserida. Considere as seguintes operações (nesta ordem) sobre essa tabela e escolha a opção que representa a tabela após estas operações (X representa DELETED e ø representa posição vazia).
    a. Inserção das chaves 3, 14, 15, 92, 65, 35 (nessa ordem)

b. Remoção da chave 15, e
c. Inserção da chave 43

6. Uma tabela hash de tamanha 10 usa endereçamento aberto, com função de hash auxiliar h(k)=k mod 10, e probing linear. Depois de inserir 6 valores em uma tabela vazia, a tabela passa a ter o estado mostrado na figura abaixo.

0	
1	
2	42
3	23
4	34
5	52
6	46
7	33
8	
9	

Qual das seguintes alternativas representa a sequência de inserções na tabela para se atingir a configuração mostrada?

a. 46, 42, 34, 52, 23, 33
b. 34, 42, 23, 52, 33, 46
c. 46, 34, 42, 23, 52, 33
d. 42, 46, 33, 23, 34, 52
e. 42, 46, 34, 23, 33, 52

7. Uma tabela hash de tamanha 10 usa endereçamento aberto, com função de hash auxiliar h(k)=k mod 10, e probing linear. De quantas formas diferentes os números 22,32,33,43,54,64 e 86 podem ser inseridos na tabela para que a tabela resultante tenha o seguinte estado final:

0	
1	
2	32
3	33
4	54
5	22
6	86
7	43
8	64
9	